

Session 13

Introduction to Version Control & Git Basics

What are you going to learn in this session?

In this session, you will understand how developers manage and track changes in their code using version control systems. You will learn why version control is essential, how Git works at a basic level, and how to start tracking your own project using simple commands.

You will also learn how to initialize a repository, stage changes, create commits, and view history. By the end of the session, you will be able to confidently manage your project versions locally using Git.

Why is this topic important?

When you work on projects, you constantly make changes to your code. Without version control, it becomes very easy to overwrite important work, lose previous versions, or break something without knowing how to fix it.

Version control solves this by maintaining a complete history of your project. It allows you to go back to previous versions, understand what changed, and collaborate effectively with others. This is not optional in the industry, it is a fundamental skill used in every development workflow.

How should you prepare yourself to attend this session?

Before attending this session, ensure your terminal is working properly and your project folder is accessible through the command line. You should be comfortable navigating directories using basic CLI commands.

If possible, install Git beforehand so that you can follow along without interruption. Keep your terminal open and be ready to execute commands step by step as demonstrated.

What should you do after the session?

After the session, you should practice initializing repositories for different folders and committing changes regularly. Try modifying files and observing how Git tracks those changes.

You should also build the habit of writing meaningful commit messages and checking history using Git commands. Over time, this will become a natural part of your development workflow.

Session 13

Class Notes

What is Version Control & Why It Matters

Version control is a system that tracks changes in your files over time. Instead of manually saving multiple versions like `file_v1`, `file_v2`, or `final_final`, version control automatically manages changes and history for you.

Without version control, you may face problems such as accidentally overwriting code, losing previous work, or struggling to collaborate with others. Version control solves these issues by keeping a detailed record of changes and allowing you to revert when needed.

Types of Version Control Systems

There are two main types of version control systems, and understanding the difference helps you see why Git is so powerful.

- Centralized VCS relies on a single central server where all code is stored, and users must connect to it to access or modify files.
- Distributed VCS allows every user to have a complete copy of the repository, including its history, making it more flexible and reliable.

Git is a distributed version control system, which means you can work locally and still have a complete project history without needing constant internet access.

Introduction to Git

Git is a tool that implements version control and helps you track changes in your project. It works locally first, which means you do not need internet access to use most of its features.

You can check if Git is installed using the following command:

```
Shell  
git --version
```

If Git is not installed, you should install it before continuing so that you can follow along with the session.

Session 13

Git Setup (One-Time Configuration)

Before using Git, you need to configure your identity so that your commits can be tracked properly. This information is attached to every change you make.

Shell

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

You can verify your configuration using:

Shell

```
git config --list
```

This step is important because it ensures that all your changes are associated with you as the author.

Git Architecture (Simple Mental Model)

Git works using three main areas, and understanding this flow is key to using it correctly. Your files move through these stages as you prepare and save changes.

- Working Directory is where your actual files exist and where you make changes.
- Staging Area is where you prepare changes before saving them permanently.
- Repository is where committed changes are stored as part of the project history.

You can think of this flow as writing, reviewing, and then saving a final version.

Initialize and Track a Project

To start using Git in a project, you need to initialize a repository inside your project folder. This creates a hidden `.git` directory that stores all version control data.

Shell

```
git init
```

You can check the status of your project using:

Session 13

```
Shell  
git status
```

This command shows which files are untracked, modified, or ready to be committed.

Add and Commit Changes (Core Workflow)

The core Git workflow involves adding changes to the staging area and then committing them to the repository. This allows you to save meaningful checkpoints in your project.

```
Shell  
git add file.txt  
git add .
```

The first command adds a specific file, while the second adds all changes in the directory.

```
Shell  
git commit -m "initial commit"
```

A commit represents a snapshot of your project at a specific point in time. Writing clear commit messages helps you understand what changes were made later.

View History

Git keeps a complete history of all commits, which you can view using the following command:

```
Shell  
git log
```

This shows details such as commit ID, author, and commit message. It allows you to track how your project has evolved over time.

.gitignore (Important Practice)

Some files should not be tracked in version control, such as dependencies or sensitive data. The .gitignore file allows you to specify which files or folders Git should ignore.

Session 13

```
Shell  
node_modules/  
.env
```

After adding entries to `.gitignore`, you can run `git status` to confirm that those files are no longer being tracked.

Common Beginner Confusions

Many learners get confused between staging and committing. Staging means preparing changes, while committing means saving those changes permanently in history.

Another common issue is files not being tracked, which usually happens when they have not been added using `git add`. Configuration issues can also arise if user details are not set correctly.

Key Takeaways

You now understand why version control is important and how Git helps you manage project changes effectively. You have learned how to initialize a repository, stage files, commit changes, and view history.

You also understand how Git organizes files into different stages and why `.gitignore` is important. With regular practice, these commands will become a natural part of your development workflow.